

AWS DevOps Deployment Assignment

Overview

This project demonstrates deployment of a **Flask backend** and **Express frontend** using different AWS approaches:

- EC2 (Single Instance)
 - EC2 (Multiple Instances)
 - ECS + ECR (Docker + Fargate)
-

Architecture & Technologies

- Docker (Containerization)
 - Amazon EC2 (Compute)
 - Amazon ECS (Fargate)
 - Amazon ECR (Container Registry)
 - VPC (Subnets, Security Groups)
 - CloudWatch (Logs)
-

Task 1: Single EC2 Deployment

Description

Both backend and frontend are deployed on a **single EC2 instance**.

Steps

- Created EC2 instance
- Installed Node.js and Python
- Ran backend on port 5000
- Ran frontend on another port
- Accessed using public IP

Access

http://54.152.165.175:3000

outputs:

```
ubuntu@ip-172-31-22-81: ~/b  × +
Collecting Flask (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask->-r requirements.txt (line 1))
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask->-r requirements.txt (line 1))
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask->-r requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask->-r requirements.txt (line 1))
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask->-r requirements.txt (line 1))
  Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from Flask->-r requirements.txt (line 1))
  Downloading werkzeug-3.1.6-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
103.4/103.4 kB 7.2 MB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
108.3/108.3 kB 11.0 MB/s eta 0:00:00
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
134.9/134.9 kB 13.4 MB/s eta 0:00:00
Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
Downloading werkzeug-3.1.6-py3-none-any.whl (225 kB)
225.2/225.2 kB 20.5 MB/s eta 0:00:00
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.3.1 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.6
(venv) ubuntu@ip-172-31-22-81:~/backend$ python app.py
Starting Flask server...
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.22.81:5000
Press CTRL+C to quit
* Restarting with stat
Starting Flask server...
* Debugger is active!
* Debugger PIN: 754-076-629
106.215.175.177 - - [21/Mar/2026 14:14:28] "GET / HTTP/1.1" 200 -
```

Frontend:



✓ Task 2: Multi EC2 Deployment

📖 Description

Backend and frontend are deployed on **separate EC2 instances**.

⚙️ Steps

- Created 2 EC2 instances
- Backend runs on one instance
- Frontend runs on another
- Connected frontend to backend using public IP

🌐 Access

- Backend:

http://<backend-ip>:5000

- Frontend:

http://<frontend-ip>

✓ Task 3: ECS + ECR Deployment

📖 Description

Application is containerized using Docker and deployed using **AWS ECS (Fargate)**.

🐳 Docker Workflow

```
docker build -t my-backend ./backend
```

```
docker build -t my-frontend ./frontend
```

```
docker tag my-backend:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/flask-backend:latest
```

```
docker tag my-frontend:latest <account-id>.dkr.ecr.us-east-1.amazonaws.com/flask-frontend:latest
```

```
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/flask-backend:latest
```

```
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/flask-frontend:latest
```

AWS Steps

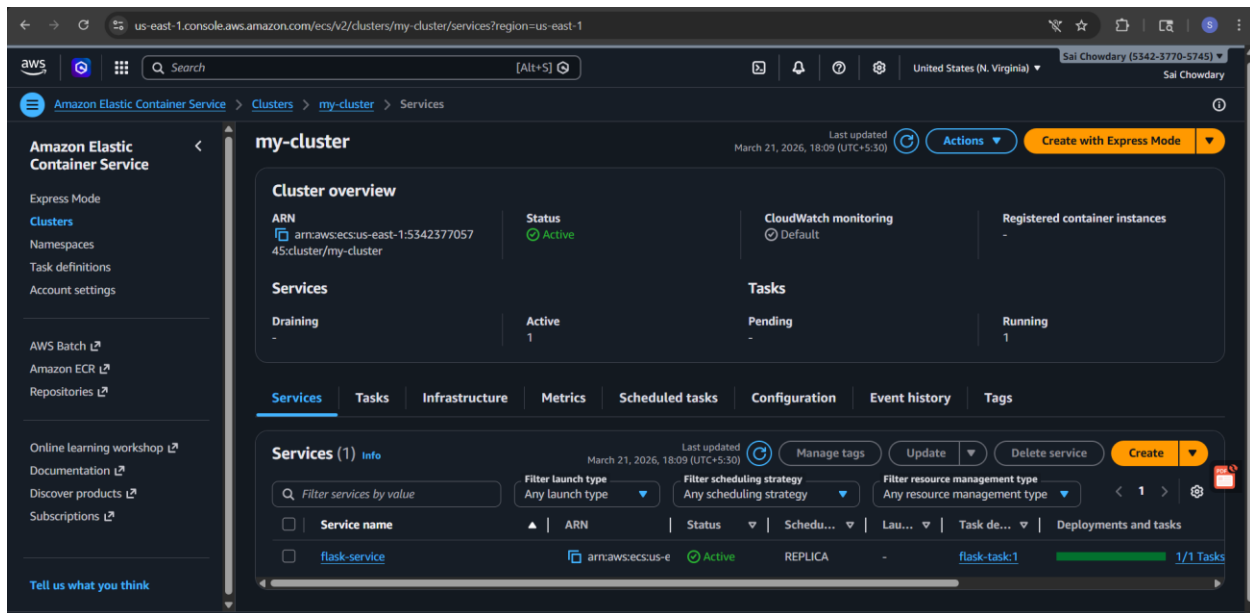
- Created ECR repositories
 - Pushed Docker images
 - Created ECS cluster (Fargate)
 - Created task definition
 - Configured networking (VPC, subnets, security group)
 - Enabled public IP
 - Deployed service
-

Access

<http://54.90.33.145:5000>

Screenshots

ECS Deployment



Application Output



⚠ Important Notes

- Flask must run with:

```
app.run(host="0.0.0.0", port=5000)
```

- Security group must allow:
 - Port 5000
 - Source: 0.0.0.0/0
 - All services are stopped after testing to avoid AWS charges.
-

Key Learnings

- Docker containerization
 - AWS ECS & ECR deployment
 - EC2 instance setup
 - Networking configuration
 - Debugging cloud issues
-

Author

Sai Kukkapalli

Conclusion

Successfully deployed a full-stack application using AWS services following DevOps best practices.